Skills that can be trusted with a migration

Devin Desktop, the Devin CLI, and an Ada → C++ worked example

30 minutes · presentation · no lab required

What this session is

A **method**, illustrated by a migration we already built and broke.

- how skills work in Devin Desktop and the Devin CLI — same format, one folder
- what separates a skill that reads well from one that holds under pressure
- the gates a migration skill needs, and the three ways round them we found
- a real defect our own test suite missed, and why

Not in this session: you typing. There is a hands-on version of this lab; this is the version for a room that cannot run our code.

The problem with "just prompt it"

A good prompt migrates one package, once, in one conversation.

A migration is:

- hundreds of packages
- the same 12 decisions, every time
- the same 7 traps, every time
- a different engineer, a different day, a fresh context window

Prompting re-derives the method every session. **Skills persist it.**

Skills in 90 seconds

- A folder with a `SKILL.md` — plus checklists, tables, templates alongside it
- Frontmatter gives it a `name` and, above all, a `description`
- Fires **automatically** when a request looks relevant, or `/name` explicitly
- Committed to the repo, so the method ships with the code

```
---
name: ada-to-cpp-migration
description: Migrate Ada (.ads/.adb) to C++17 with parity proven against
  captured reference output. Use for any port/translate/rewrite request.
---
```

Devin Local is the agent in Desktop since 3.9.19, when Cascade was removed —
so `/name`, not `@name`.

Desktop and CLI: one skill, two front doors

	Where it lives
Project	`.agents/skills/<name>/` — committed, shared with the team
Global	`~/.config/devin/skills/<name>/` · `%APPDATA%\devin\skills\<name>\`

Same folder layout, same frontmatter, same `/name` invocation, whether you are in the desktop app or the terminal.

The practical consequence: **author it in the repo**. A skill in your home directory helps you; a skill in `.agents/skills/` helps everyone who clones, and gets reviewed like code.

Progressive disclosure

Until the skill fires, the model sees **only** `name` and `description`.

1. The description is not a summary — it is a **trigger**. Write it as *when to invoke me*, in the words a real request will use.
2. `SKILL.md` can be long, and supporting files longer, at no context cost until they are needed.
3. A skill nobody triggers is a file nobody reads.

The commonest defect in a hand-written skill is not bad procedure. It is a description that never matches how anyone asks.

Skill vs rule

	Loaded	Use for
Rule (`AGENTS.md`)	always-on project guidance	short, universal constraints
Skill	on demand, when relevant	procedures with judgement + resources

A migration method is a skill: it is long, it is conditional, it carries supporting files, and you want it to fire even when the engineer forgets it exists.

The worked example

```
Telemetry             constrained subtypes, decimal fixed point, Image
Telemetry.Sensors     abstract tagged type, dispatching, access-to-class
Telemetry.Parsing     Text_IO, string slices, exceptions with messages
Telemetry.Stats       generic subprogram, integer-scaled aggregation
Telemetry.Alerts      discriminated record with a variant part
```

~450 lines of Ada: CSV of sensor readings → calibrate → summary → alerts.

Chosen because every construct here has a *plausible wrong* C++ translation — one that compiles cleanly and gives different answers.

Compilation is not migration

The question a migration has to answer is not "does it build" but **"does it still do the same thing"**.

Before a line of C++ existed we ran the Ada program and captured, per case:

```
stdout        stderr        exit status
```

`ctest` runs the C++ binary on the same input and compares all three, byte for byte. That is the definition of done — not a code review, not a diff.

> The captured output is the specification for the behaviour these cases cover: characterization evidence, not proof of equivalence.

The gates: prohibitions, not preferences

- never edit the captured reference output
- never edit the legacy sources
- never weaken, skip or delete a parity test
- never add a tolerance to a comparison that was exact
- never translate a fixed point or integer type to floating point
- never drop a run-time constraint check
- never hard-code fixtures or expected output into the program
- never read or generate from the captured output at build or run time
- never wrap, link to or shell out to the legacy program to pass a case
- if it cannot be translated faithfully — **stop and report**

Why *never* and not *prefer*

An instruction competes with everything else in context.
A prohibition is checkable — by the model, and by you in review.

Decoration	Gate
"Prefer not to modify golden files"	"Never edit anything in `golden/`"
"Try to keep tests passing"	"Never report done while any parity case fails or was skipped"
"Be careful with numeric types"	"Never translate fixed point to `double`"

If you cannot write it as *never* or *stop and report*, it is guidance — and guidance belongs in a reference file, not in the gate list.

The gate that deadlocks

"Never advance while any parity case fails" reads like rigour.

The suite is end-to-end. It stays red until the **last** package lands — so a literal-minded agent refuses to start package two.

When	Gate
after each package	builds clean · nothing that passed now fails
dependency closure complete	fixing red cases is the only work allowed
completion	never report done while any case fails or was skipped

You find this by **running** the skill, not by reading it. Every gate list you write has one of these in it.

Three ways to a green suite with nothing migrated

The agent can read the inputs *and* the expected output. So:

1. **Special-case the fixture** — branch on the filename, print the answer.
2. **Copy the evidence** — `std::ifstream in("golden/report.stdout");` `std::cout << in.rdbuf();` — hard-codes nothing, so a no-hard-coding rule misses it. Forbid build time too, or a generated header is next.
3. **Delegate** — a C++ binary that shells out to the Ada one. Every case green, nothing translated.

Each needed its own prohibition. Assume your list is one loophole short.

Prose gates vs enforced gates

```
permissions:
  deny:  [Write(golden/**), Write(ada/**)]
  ask:   [Write(cpp/tests/**)]
```

`deny` holds **whether or not** the model agrees with you.

Tests are `ask`, not `deny`: adding a parity case is legitimate work,
weakening one is not, and no path pattern can tell those apart.
Three layers, not two: prose rule → tool-level guardrail → OS isolation.

> Enforce what the platform can enforce. Reserve prose for judgement — "do not map fixed point to `double`" fits no pattern; it is about meaning.

The traps are the whole job

Ada	C++
`package P` / child `P.C`	`namespace p` / `p::c`, header per spec
abstract tagged type	abstract class + virtual, `override`
discriminated record, variant part	`std::variant` of payload structs
generic package/subprogram	template
`array (Positive range <>)`	`std::vector` — **trap**: 1-based
`raise E with "msg"`	`throw` — **trap**: the text is observable output
`Integer'Image`	**trap**: leading blank on non-negatives
`delta 0.1 digits 6`	**trap**: exact decimal — *not* `double`

Traps go in a reference file the skill loads when needed, not in `SKILL.md`.

The defect our own suite missed

```
type Celsius is delta 0.1 digits 6 range -80.0 .. 150.0;
```

Our finished C++ checked that range **on parsed input only**. A reading of 150.0 on a sensor with a +1.5 calibration offset produced 151.5 and printed a cheerful report; Ada raises `Constraint_Error` and exits 2.

Three parity cases all passed. The migration was wrong, and the answer key was breaking its own "never drop a run-time constraint check" rule.

Two cases now cover it: a constraint that fails on a **computed** value, and a missing input file — a failure *before* parsing, raising an exception that is not the program's own.

What that story is actually about

- The gate was right. The **evidence** was too thin to catch its violation.
- Coverage gaps do not look like gaps from the inside. They look green.
- So pick cases by **failure mode**, not by counting: normal, malformed, boundary, and at least one that fails *after* validation.
- And re-read your gates against your suite: a rule no case can falsify is a rule you are trusting on faith.

Monday morning, on your own codebase

1. Pick **one runnable slice** — not the system. Verify its build and run commands today.
2. Capture stdout, stderr and exit status for 4–5 cases. If behaviour is not deterministic, making it deterministic **is** task one.
3. Write `SKILL.md`: description as trigger, numbered loop, gate list.
4. Attack it in a fresh conversation — ask it to relax a type, edit the evidence, special-case a fixture. A refusal is the passing result.
5. Add `permissions` for what the platform can enforce; commit it.

The three things

1. The **description** decides whether the skill exists in practice.
2. The **gates** decide whether its output can be trusted — enforce the ones the platform can enforce, and assume the list is incomplete.
3. The **evidence** decides whether the migration is real — and how good the evidence is decides what the gates are worth.